

1. Title of the Mini Project

Project Title: Smart Route Optimization System Using Bellman Ford Algorithm for Shortest Path Detection and Traffic Analysis

2. Abstract

This mini project focuses on the design and implementation of a Smart Route Optimization System using the Bellman Ford Algorithm, an important shortest path algorithm in graph theory. The main objective of this project is to find the shortest and optimized routes between cities by analyzing weighted routes in a graph. The system is developed using the C programming language and provides shortest path calculation and route optimization between cities.

The project demonstrates:

- City network representation
- Optimized shortest route
- Minimum distance calculation
- Shortest path generation
- Negative weight cycle detection

The Bellman Ford Algorithm is capable of handling weighted graphs and detecting negative edge weights, making it suitable for real world routing systems and network optimization problems.

The implementation includes modular C code with separate functions for graph representation, shortest path calculation, path generation, and route display.

Time complexity is analyzed as:

- Best Case: $O(VE)$
- Average Case: $O(VE)$
- Worst Case: $O(VE)$

Space complexity is determined to be:

- $O(V)$

Applications in navigation systems, computer networks, traffic management systems, Artificial Intelligence based routing systems, and smart transportation systems are also explored.

Keywords: Bellman Ford Algorithm, Time Complexity, Space Complexity, Graphs and Adjacency Lists, Route Optimization and Traffic Management

3. Introduction

3.1 Background and Motivation

Algorithms are the backbone of computer science, providing systematic procedures for solving computational problems. Graph algorithms are widely used in navigation systems, network routing, GPS applications, and smart transportation systems. Shortest path optimization helps determine efficient travel routes and reduces traffic congestion.

Shortest path algorithms help determine the minimum distance or lowest cost path between two locations. Efficient route optimization helps save travel time, fuel, and transportation costs while improving traffic management and user convenience.

Bellman Ford Algorithm was first introduced by Richard Bellman and Lester Ford in 1958. It belongs to the class of [divide and conquer / greedy / dynamic programming / backtracking / etc.] algorithms and is widely used in network routing and shortest path analysis.

3.2 Problem Statement

Problem Statement:

Given cities connected through weighted routes, design and implement the Bellman Ford Algorithm to determine the shortest and optimized path from a source city to all other cities.

Input:

Input includes cities, routes, and distance weights between connected cities.

Output:

Display the shortest route, minimum distance, and optimized path from the selected source city.

Constraints:

Number of cities ≥ 1 and graph edges must contain valid weights. Negative weight cycles are not allowed in the graph.

3.3 Objectives

- To understand the working principle of the Bellman Ford Algorithm.
- To implement the algorithm in C programming language using modular code.
- To analyze time and space complexity of the algorithm.
- To determine shortest and optimized routes between cities.
- To test the algorithm on normal and extreme cases.
- To study real world applications in routing and navigation systems.

3.4 Scope of the Project

This project focuses on developing and visualizing a smart route optimization system using the Bellman Ford Algorithm. The system represents cities and routes using graph theory concepts and helps determine the shortest and most optimized path between cities connected through weighted routes.

The project is mainly designed for educational and demonstration purposes to understand graph algorithms and shortest path optimization techniques in a simple and interactive manner. The main aim of the project is to study the working of the Bellman Ford Algorithm and demonstrate how shortest path optimization can be applied in route planning systems. The output of the project is displayed through a console based interface where users can select a source city and view the shortest routes and minimum distances to all other cities.

The project includes:

- shortest path calculation
- route optimization
- graph based route analysis
- optimized path generation using C programming language.

The project does not include:

- Real-time GPS integration
- Live internet traffic APIs
- Large-scale cloud deployment
- large scale distributed routing systems

4. Implementation with Comments

4.1 Programming Language and Environment

Language	C Programming
IDE / Editor/Compiler	Visual Studio Code / GCC Compiler
OS / Platform	Windows 10
Libraries Used	stdio.h
Github Link	

4.2 Algorithm Design

ALGORITHM Bellman Ford Algorithm

INPUT

- Cities represented as vertices
- Weighted routes between cities
- Source city

OUTPUT

- Shortest distance from source city
- Optimized route path

FUNCTION BellmanFord(edges, source)

FOR each vertex

 distance[vertex] $\leftarrow$ infinity

 parent[vertex] $\leftarrow$ -1

END FOR

distance[source] $\leftarrow$ 0

FOR i $\leftarrow$ 1 TO V - 1

 FOR each edge (u, v, weight)

 IF distance[u] + weight < distance[v]

 distance[v] $\leftarrow$ distance[u] + weight

 parent[v] $\leftarrow$ u

 END IF

 END FOR

END FOR

FOR each edge (u, v, weight)

 IF distance[u] + weight < distance[v]

 PRINT "Negative Weight Cycle Detected"

 RETURN

 END IF

END FOR

DISPLAY shortest distance and optimized path

END FUNCTION

4.3 Source Code with Inline Comments (Font: Consolas- ,size-11)

```
# =====
# Mini Project : Smart Route Optimization System
# Algorithm   : Bellman Ford Algorithm
# Author      : [Zainab Unnisa] | USN : [R24EJ163]
# Date       : [27-05-2026]
# Description : Finds shortest optimized route between cities
#              using Bellman Ford Algorithm.
# =====

#include <stdio.h>
#define V 7
#define E 10
#define INF 99999

// Structure for graph edge
struct Edge {

    int source;
    int destination;
    int weight;
};

// Function to print shortest path
void printPath(int parent[], int vertex) {

    if(parent[vertex] == -1) {

        printf("%d ", vertex);
        return;
    }

    printPath(parent, parent[vertex]);

    printf("-> %d ", vertex);
}

// Bellman Ford Algorithm
void bellmanFord(struct Edge edges[], int source) {

    int distance[V];
    int parent[V];

    // Initialize distances
    for(int i = 0; i < V; i++) {

        distance[i] = INF;
        parent[i] = -1;
    }

    distance[source] = 0;
```

```
// Relax edges
```

```
for(int i = 1; i <= V - 1; i++) {  
  
    for(int j = 0; j < E; j++) {  
  
        int u = edges[j].source;  
        int v = edges[j].destination;  
        int w = edges[j].weight;  
  
        if(distance[u] != INF &&  
            distance[u] + w < distance[v]) {  
  
            distance[v] = distance[u] + w;  
            parent[v] = u;  
        }  
    }  
}
```

```
// Negative cycle detection
```

```
for(int j = 0; j < E; j++) {  
  
    int u = edges[j].source;  
    int v = edges[j].destination;  
    int w = edges[j].weight;  
  
    if(distance[u] != INF &&  
        distance[u] + w < distance[v]) {  
  
        printf("\nNegative Weight Cycle Detected\n");  
        return;  
    }  
}
```

5. Modules Identified

The implementation has been divided into the following logical modules for clarity, reusability, and ease of testing:

#	Module Name	Description	Function(s) / Class(es)
1	Input Module	Reads source city from user	main()
2	Graph Module	Stores city routes and weights	Edge Structure
3	Bellman Ford Module	Finds shortest distances	bellmanFord()
4	Path Module	Displays optimized route path	printPath()
5	Output Module	Displays shortest route details	bellmanFord()

6. Output with Screenshots

This section presents the output of the Smart Route Optimization System for different test cases including normal, edge, and extreme conditions.

6.1 Test Case Strategy

All test cases have been categorized into:

- Normal Cases – Valid city routes with expected shortest paths.
- Boundary / Edge Cases – Same source and destination, invalid or unavailable routes.
- Extreme Cases – Negative weights, large path values, and complex route combinations.

6.2 Test Case Table

TC No	Test Category	Input	Expected Output	Actual Output	Status
TC-01	Normal	Source Vertex = 0	Shortest routes from Bangalore	Shortest routes displayed	PASS
TC-02	Normal	Source Vertex = 1	Optimized paths from Hyderabad	Optimized paths displayed	PASS
TC-03	Boundary	Source Vertex = 6	Route calculation for Kolkata	Correct distances displayed	PASS
TC-04	Boundary	Invalid Vertex	Error or invalid input handling	Invalid input handled	PASS
TC-05	Extreme	Weighted graph with multiple paths	Minimum distance route	Correct shortest path displayed	PASS
TC-06	Extreme	Graph with negative edge	Proper shortest distance calculation	Correct output displayed	PASS
TC-07	Extreme	Graph with negative cycle	Negative cycle detection	Negative cycle detected	PASS

6.3 Screenshots of Output

```

City Mapping
=====
0 -> Bangalore
1 -> Hyderabad
2 -> Chennai
3 -> Mumbai
4 -> Delhi
5 -> Pune
6 -> Kolkata

Enter Source Vertex (0 - 6) : 0

=====
Shortest Route Details
=====

Destination : Bangalore
Distance    : 0 km
Path        : 0

Destination : Hyderabad
Distance    : 3 km
Path        : 0 -> 2 -> 1

Destination : Chennai
Distance    : 2 km
Path        : 0 -> 2

Destination : Mumbai
Distance    : 5 km
Path        : 0 -> 2 -> 1 -> 3

Destination : Delhi
Distance    : 9 km
Path        : 0 -> 2 -> 1 -> 4

Destination : Pune
Distance    : 8 km
Path        : 0 -> 2 -> 1 -> 3 -> 5

Destination : Kolkata
Distance    : 14 km
Path        : 0 -> 2 -> 1 -> 4 -> 6

```

Screenshot 6.3.1 – Normal Case Output, Multiple Route Optimization Case


```

=====
City Mapping
=====
0 -> Bangalore
1 -> Hyderabad
2 -> Chennai
3 -> Mumbai
4 -> Delhi
5 -> Pune
6 -> Kolkata

Enter Source Vertex (0 - 6) : 1

=====
Shortest Route Details
=====

Destination : Bangalore
Distance    : No Route Available
Path       : No Path

Destination : Hyderabad
Distance    : 0 km
Path       : 1

Destination : Chennai
Distance    : No Route Available
Path       : No Path

Destination : Mumbai
Distance    : 2 km
Path       : 1 -> 3

Destination : Delhi
Distance    : 6 km
Path       : 1 -> 4

Destination : Pune
Distance    : 5 km
Path       : 1 -> 3 -> 5

Destination : Kolkata
Distance    : 11 km
Path       : 1 -> 4 -> 6

```

Screenshot 6.3.2 – Edge Case: Same Source and Destination Case

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
=====
City Mapping
=====
0 -> Bangalore
1 -> Hyderabad
2 -> Chennai
3 -> Mumbai
4 -> Delhi
5 -> Pune
6 -> Kolkata

Enter Source Vertex (0 - 6) : 6

=====
Shortest Route Details
=====

Destination : Bangalore
Distance    : No Route Available
Path       : No Path

Destination : Hyderabad
Distance    : No Route Available
Path       : No Path

Destination : Chennai
Distance    : No Route Available
Path       : No Path

Destination : Mumbai
Distance    : 4 km
Path       : 6 -> 3

Destination : Delhi
Distance    : 9 km
Path       : 6 -> 3 -> 5 -> 4

Destination : Pune
Distance    : 7 km
Path       : 6 -> 3 -> 5

Destination : Kolkata
Distance    : 0 km
Path       : 6

```

Screenshot 6.3.3 – Edge Case: No Route Available Case

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=====
City Mapping
=====
0 -> Bangalore
1 -> Hyderabad
2 -> Chennai
3 -> Mumbai
4 -> Delhi
5 -> Pune
6 -> Kolkata

Enter Source Vertex (0 - 6) : 4

=====
Shortest Route Details
=====

Destination : Bangalore
Distance    : No Route Available
Path        : No Path

Destination : Hyderabad
Distance    : No Route Available
Path        : No Path

Destination : Chennai
Distance    : No Route Available
Path        : No Path

Destination : Mumbai
Distance    : 9 km
Path        : 4 -> 6 -> 3

Destination : Delhi
Distance    : 0 km
Path        : 4

Destination : Pune
Distance    : 12 km
Path        : 4 -> 6 -> 3 -> 5

Destination : Kolkata
Distance    : 5 km
Path        : 4 -> 6
```

Screenshot 6.3.4 – Negative Weight Edge Case

```
PS D:\zainab\daaprrject> gcc bellman.c -o bellman
PS D:\zainab\daaprrject> .\bellman.exe

=====
City Mapping
=====
0 -> Bangalore
1 -> Hyderabad
2 -> Chennai
3 -> Mumbai
4 -> Delhi
5 -> Pune
6 -> Kolkata

Enter Source Vertex (0 - 6) : 0

Negative Weight Cycle Detected
PS D:\zainab\daaprrject> █
```

Screenshot 6.3.5– Negative Cycle Detection

7. Time and Space Complexity Analysis

7.1 Time Complexity

The time complexity of the Bellman Ford Algorithm is analyzed for three cases:

Metric	Best Case	Average Case	Worst Case
Time Complexity	$O(VE)$	$O(VE)$	$O(VE)$
Space Complexity	$O(V)$	$O(V)$	$O(V)$

Where:

V represents the number of vertices

E represents the number of edges

7.2 Detailed Analysis

- **Best Case – $O(VE)$:** Even if the shortest path is found early, the Bellman Ford Algorithm still relaxes all edges $V - 1$ times. Therefore, the best case time complexity remains $O(VE)$.
Example: If Bangalore directly connects to Chennai with the shortest distance, the algorithm still checks all other routes repeatedly.
- **Average Case – $O(VE)$:** [All edges are repeatedly checked and updated to calculate the shortest distance from the source city to all other cities. Example: The algorithm checks routes like Bangalore $\rightarrow$ Hyderabad and Bangalore $\rightarrow$ Chennai multiple times to determine the minimum distance.]
- **Worst Case – $O(VE)$:** [The worst case occurs when distance values continue updating during every iteration. Since all E edges are processed for $V - 1$ iterations, the worst case complexity is $O(VE)$. Example: If multiple routes between cities keep producing shorter distances, the algorithm must continue relaxing edges until the final iteration.]

7.3 Space Complexity

The Bellman Ford Algorithm requires additional memory for storing distance and parent arrays during shortest path calculation. Therefore, it is considered an out of place algorithm because extra space is used proportional to the number of vertices.

- **In place:** The algorithm does not work completely in place because additional arrays are required for computation.
- **Out of place:** Arrays such as `distance[V]` and `parent[V]` are used to store shortest distances and optimized paths. Hence, the space complexity is $O(V)$.
- **Recursive overhead:** The `printPath()` function uses recursion to display the optimized route, which creates a small call stack overhead depending on the path length.

7.4 Amortized Complexity Analysis

Amortized analysis provides the average cost per operation over a sequence of operations, rather than analyzing each operation in isolation.

Aggregate Method:

The total cost of relaxing all edges repeatedly for $V - 1$ iterations is $O(VE)$. Since the algorithm processes all E edges during each iteration, the average cost per iteration remains proportional to $O(E)$.

Total Cost for all iterations:

$$T(n) = O(VE)$$

Amortized Cost per iteration:

$$T(n) / V = O(E)$$

Thus, the amortized cost per iteration in the Bellman Ford Algorithm is $O(E)$. Although amortized analysis is less significant for Bellman Ford compared to dynamic data structures, the Aggregate Method helps explain the repeated edge relaxation process.

8. Can the Algorithm Be Optimized?

Yes, the Bellman Ford Algorithm can be optimized in terms of execution time and memory efficiency for specific routing applications. Although the theoretical time complexity remains $O(VE)$, certain improvements can reduce unnecessary computations and improve overall performance.

8.1 Current Limitations

- The Bellman Ford Algorithm takes more time to execute compared to some faster shortest path algorithms such as Dijkstra's Algorithm.
- The algorithm repeatedly checks all routes multiple times, which increases computation time for large graphs.
- Performance may decrease when the number of cities and routes becomes very large.
- Extra memory is required to store shortest distances and parent paths for all cities.

8.2 Optimization Strategies

Optimization Strategy	Data Structure Used	Improvement Achieved
Early stopping technique	Arrays	Reduces unnecessary edge relaxations
Adjacency list representation	Linked List / Array List	Improves memory efficiency
Dijkstra's Algorithm for positive weights	Priority Queue / Min Heap	Faster execution
Efficient graph traversal	Graph Structure	Reduces computation overhead
Priority queue based processing	Heap / Priority Queue	Improves route processing efficiency

9. Applications of the Algorithm

9.1 Applications in Computer Science

1 Navigation Systems - Bellman Ford Algorithm is used in GPS and route optimization systems to determine the shortest path between locations.

2 Computer Network Routing - The algorithm is used in routing protocols to find efficient data transmission paths in computer networks.

3 Traffic Management Systems - Shortest path algorithms help reduce traffic congestion by identifying optimized travel routes.

4 Transportation Planning - Used in smart transportation systems for efficient route analysis and travel planning.

5 Delivery and Logistics Systems - Helps logistics companies determine minimum distance delivery routes to reduce transportation cost.

6 Smart City Applications - Used in smart city infrastructure for efficient road network analysis and traffic control.

9.2 Applications in Artificial Intelligence (AI)

1 Intelligent Route Planning - AI based navigation systems use shortest path algorithms to determine optimized travel routes.

2 Autonomous Vehicle Navigation - Self driving vehicles use route optimization algorithms for efficient and safe navigation.

3 AI Based Traffic Prediction - AI systems analyze traffic conditions and use shortest path algorithms to reduce congestion.

4 Smart Transportation Systems - Bellman Ford Algorithm helps AI systems optimize traffic flow.

9.3 Applications in Machine Learning (ML)

1 Traffic Prediction Models - Machine Learning models use route optimization data for predicting traffic conditions.

2 Route Recommendation Systems - ML systems recommend efficient travel routes based on shortest path analysis.

3 Logistics Optimization - Shortest path algorithms help optimize delivery routes and transportation planning in ML based systems.

10. Conclusion

In this mini project, we successfully designed, implemented, and analyzed the Bellman Ford Algorithm as part of the Design and Analysis of Algorithms course. The implementation was carried out in C programming language using well structured and modular code with detailed inline comments for better readability and understanding.

Through this project, we gained a clear understanding of the working mechanism of graph based shortest path algorithms and route optimization techniques. The Bellman Ford Algorithm repeatedly relaxes graph edges to determine the shortest distance from a source city to all other cities in a weighted graph. The time complexity of the algorithm was determined to be $O(VE)$ in the best case, average case, and worst case, while the space complexity was found to be $O(V)$. Amortized analysis was applied using the Aggregate Method, where the amortized cost per iteration was proportional to $O(E)$.

The testing phase verified the correctness of the implementation across normal cases, boundary conditions, and extreme cases, all of which produced expected outputs. We also explored optimization opportunities using adjacency list representation and efficient graph traversal techniques to improve memory efficiency and reduce unnecessary computations.

The Bellman Ford Algorithm has important applications in computer networks, navigation systems, traffic management, Artificial Intelligence based route planning, and Machine Learning based transportation systems. Overall, this project helped us understand the importance of algorithm analysis in developing efficient and scalable route optimization systems.

10.1 Future Enhancements

- Implement a graphical user interface for better visualization and user interaction.
- Extend the system to support real time traffic and GPS based route analysis.
- Develop a larger scale routing system capable of handling more cities and complex road networks.
- Integrate the algorithm with map based visualization tools for smart navigation systems.
- Improve performance using advanced data structures such as priority queues and adjacency lists.

11. References

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

S. Dasgupta, C. H. Papadimitriou, and U. Vazirani, *Algorithms*. New York, NY, USA: McGraw Hill, 2006.

A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*. Reading, MA, USA: Addison Wesley, 1974.

R. Bellman, "On a Routing Problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

L. R. Ford Jr., and D. R. Fulkerson, *Flows in Networks*. Princeton, NJ, USA: Princeton University Press, 1962.

GeeksforGeeks, "Bellman Ford Algorithm," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23/>. [Accessed: 21-May-2026].

Declaration

We, the undersigned students, hereby declare that the mini project report titled "[Project Title]" submitted to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the 4th Semester Design and Analysis of Algorithms course is our original work. The project has been carried out by us under the guidance of [Guide Name] and has not been submitted elsewhere for any other examination or award.

Student Name	Zainab Unnisa Signature: _____
Student SRN	R24EJ163
Faculty	Prof.DivyaShree / Dr.Aravindhan K Signature: _____
Date	27/05/2026